# torch.func.grad#

https://pytorch.org/docs/stable/generated/torch.func.grad.html

**Description:**

grad operator helps computing gradients of func with respect to the input(s) specified by argnums. This operator can be nested to compute higher-order gradients. func (Callable) – A Python function that takes one or more arguments. Must return a single-element Tensor. If specified has_aux equals True, function can return a tuple of single-element Tensor and other auxiliary objects: (output, aux). argnums (int or Tuple[int]) – Specifies arguments to compute gradients with respect to. argnums can be single integer or tuple of integers. Default: 0. has_aux (bool) – Flag indicating that func returns a tensor and other auxiliary objects: (output, aux). Default: False. Function to compute gradients with respect to its inputs. By default, the output of the function is the gradient tensor(s) with respect to the first argument. If specified has_aux equals True, tuple of gradients and output auxiliary objects is returned. If argnums is a tuple of integers, a tuple of output gradients with respect to each argnums value is returned.

## Function Signature

```
torch.func.grad(func, argnums=0, has_aux=False)[source]#
Parameters
Returns
Return type
```

**Returns:**

Callable

## Code Examples

```
>>> from torch.func import grad
>>> x = torch.randn([])
>>> cos_x = grad(lambda x: torch.sin(x))(x)
>>> assert torch.allclose(cos_x, x.cos())
>>>
>>> # Second-order gradients
>>> neg_sin_x = grad(grad(lambda x: torch.sin(x)))(x)
>>> assert torch.allclose(neg_sin_x, -x.sin())
```

```
>>> from torch.func import grad, vmap
>>> batch_size, feature_size = 3, 5
>>>
>>> def model(weights, feature_vec):
>>> # Very simple linear model with activation
>>>     assert feature_vec.dim() == 1
>>>     return feature_vec.dot(weights).relu()
>>>
>>> def compute_loss(weights, example, target):
>>>     y = model(weights, example)
>>>     return ((y - target) ** 2).mean()  # MSELoss
>>>
>>> weights = torch.randn(feature_size, requires_grad=True)
>>> examples = torch.randn(batch_size, feature_size)
>>> targets = torch.randn(batch_size)
>>> inputs = (weights, examples, targets)
>>> grad_weight_per_example = vmap(grad(compute_loss), in_dims=
(None, 0, 0))(
...     *inputs
... )
```

```
>>> from torch.func import grad
>>> def my_loss_func(y, y_pred):
>>>    loss_per_sample = (0.5 * y_pred - y) ** 2
>>>    loss = loss_per_sample.mean()
>>>    return loss, (y_pred, loss_per_sample)
>>>
>>> fn = grad(my_loss_func, argnums=(0, 1), has_aux=True)
>>> y_true = torch.rand(4)
>>> y_preds = torch.rand(4, requires_grad=True)
>>> out = fn(y_true, y_preds)
>>> # > output is ((grads w.r.t y_true, grads w.r.t y_preds),
(y_pred, loss_per_sample))
```

```
>>> def f(x):
>>>     with torch.no_grad():
>>>         c = x ** 2
>>>     return x - c
```

```
>>> with torch.no_grad():
>>>     grad(f)(x)
```

## Notes & Warnings

NOTE

Note Using PyTorch torch.no_grad together with grad. Case 1: Using torch.no_grad inside a function: >>> def f(x): >>> with torch.no_grad(): >>> c = x ** 2 >>> return x - c In this case, grad(f)(x) will respect the inner torch.no_grad. Case 2: Using grad inside torch.no_grad context manager: >>> with torch.no_grad(): >>> grad(f)(x) In this case, grad will respect the inner torch.no_grad, but not the outer one. This is because grad is a “function transform”: its result should not depend on the result of a context manager outside of f.

## Detailed Documentation

### Documentation

grad operator helps computing gradients of func with respect to the input(s) specified

by argnums. This operator can be nested to compute higher-order gradients.

func (Callable) – A Python function that takes one or more arguments. Must return a single-element Tensor. If specified has_aux equals True, function can return a tuple of single-element Tensor and other auxiliary objects: (output, aux).

argnums (int or Tuple[int]) – Specifies arguments to compute gradients with respect to. argnums can be single integer or tuple of integers. Default: 0.

has_aux (bool) – Flag indicating that func returns a tensor and other auxiliary objects: (output, aux). Default: False.

Function to compute gradients with respect to its inputs. By default, the output of the function is the gradient tensor(s) with respect to the first argument. If specified has_aux equals True, tuple of gradients and output auxiliary objects is returned. If argnums is a tuple of integers, a tuple of output gradients with respect to each argnums value is returned.

Example of using grad:

When composed with vmap, grad can be used to compute per-sample-gradients:

Example of using grad with has_aux and argnums:

Using PyTorch torch.no_grad together with grad.

Case 1: Using torch.no_grad inside a function:

In this case, grad(f)(x) will respect the inner torch.no_grad.

Case 2: Using grad inside torch.no_grad context manager:

In this case, grad will respect the inner torch.no_grad, but not the outer one. This is

because grad is a “function transform”: its result should not depend on the result of a context manager outside of f.

---